



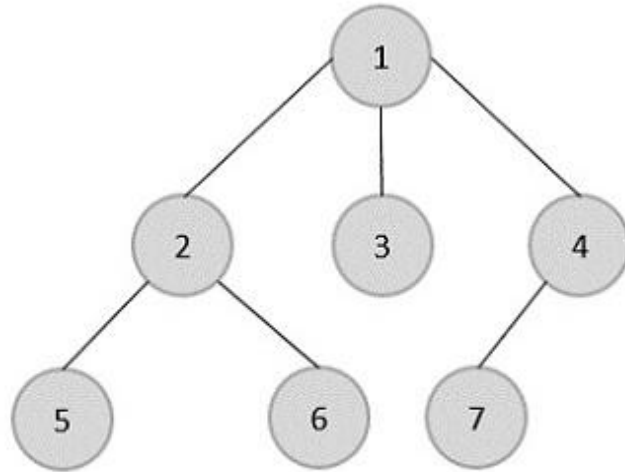
Tree Basic

Abdullah All Mamun Siam

Lecturer, Dhaka International University(DIU)

Tree Data Structure

A tree is a non-linear abstract data type with a hierarchy-based structure. It consists of nodes that are connected via links.



General Tree Data Structure

Why Use a Tree Data Structure?

Linear data structures like arrays, linked lists, stacks, and queues store data sequentially. As the data size grows, the time complexity for performing operations also increases, which is often inefficient in modern computational scenarios.

In contrast, tree data structures, being non-linear, enable faster and more efficient access to data, making them a better choice for handling large datasets.

Tree Terminologies

Node

A node is a fundamental unit used to represent data and its relationships with other nodes. It is a building block of non-linear data structures like trees and graphs. A node is an entity that contains a key or value and pointers to its child nodes.

Edge

It is the link between any two nodes. They help illustrate how nodes are connected and how data flows between them.

Edges can be:

- **Directed:** The connection has a direction, such as in a one-way road.
- **Undirected:** The connection has no direction, such as in a two-way road.

Tree Terminologies

Weighted or Unweighted:

- Weighted edges have an associated cost or value, such as distance or time.
- Unweighted edges have no such value.

Root Node

The root node is the topmost node in a hierarchical data structure, such as a tree, and serves as the starting point for accessing all other nodes in the structure. It is the only node in the tree that does not have a parent. There is only one root per tree and one path from the root node to any node.

Tree Terminologies

Parent Node

A parent node in a tree data structure is a node that has one or more child nodes connected to it. It acts as an immediate predecessor to its children, forming a hierarchical relationship within the tree.

Child Node

A child node in a tree data structure is a node that is directly connected to and descends from another node, referred to as its parent node.

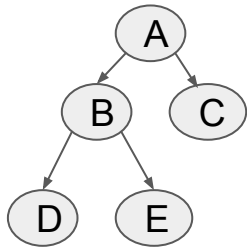
Tree Terminologies

Leaf

The node which does not have any child node is called the leaf node.

Degree of a Node

The degree of a node in a tree or graph is the total number of connections (or edges) that the node has. It reflects how many child nodes a node has in a tree, or how many direct neighbors it has in a graph.



Node **A** has a degree of **2** (children: B, C).

Node **B** has a degree of **2** (children: D, E).

Node **C**, **D**, and **E** each have a degree of **0** (no children).

Tree Terminologies

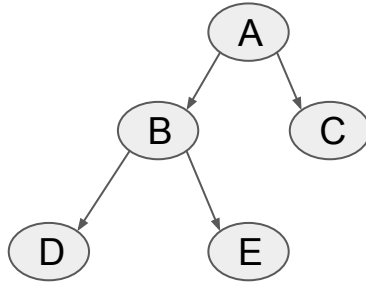
In a **directed graph**:

In-degree:

The number of edges coming into the node.

Out-degree:

The number of edges going out of the node.



Tree Terminologies

Node A:

In-degree: 0

Out-degree: 2 (edge from A $\rightarrow$ B,C).

Node B:

In-degree: 1 (edges from A $\rightarrow$ B).

Out-degree: 2 (edge from B $\rightarrow$ D,E).

Node C:

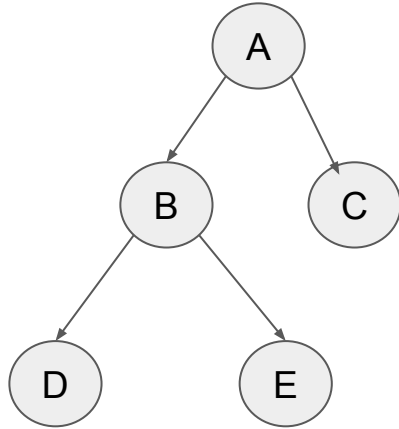
In-degree: 1 (edges from A $\rightarrow$ C).

Out-degree: 0

Tree Terminologies

Height of a Node

The height of a node in a tree data structure is the length of the longest path from that node to a leaf.



Node A: Height = 2

Node B: Height = 1

Node C: Height = 0

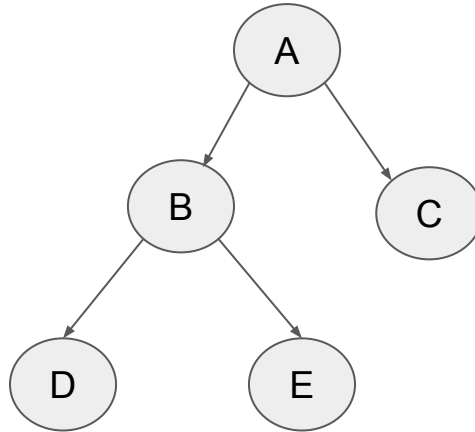
Node D: Height = 0

Node E: Height = 0

Tree Terminologies

Level

In a tree data structure, the **level** of a node represents its position in the hierarchy, defined by the distance (in terms of edges) from the **root node**. The root node is at **level 0**, and as you move downward, the level increases by 1 for each step.



Level 0: Node A (root node).

Level 1: Nodes B, C (children of A).

Level 2: Nodes D, E (children of B).

Applications of Trees

Trees are widely used in computer science and real-world applications due to their hierarchical structure and efficient operations. Here are some common applications of trees:

- 1.Data Organization and Representation
- 2.Searching and Sorting
- 3.Graphical User Interfaces (GUIs)
- 4.Routing and Network Applications
- 5.Artificial Intelligence and Machine Learning
- 6.Compiler Design
- 7.Compression and Encoding
- 8.Games and Simulations